HONEYWELL SOFTWARE ENGINEERING TEAMS — IA · BA · PA&T

AI Champions Programme — GitHub Copilot Enterprise, SDD, MCP & Agentic Software Engineering

# Module 05

## SDD Across the Full SDLC: Greenfield, Brownfield, Modernization

*Carry one habit — specs as the contract — across three terrains: build new apps end to end, change existing code without breaking it, and modernize legacy systems one safe slice at a time.*

DURATION
**3.5 hours**

FORMAT
**3 concepts · 2 labs · 1 demo**

PRIMARY TOOL
**Copilot + Spec Kit**

# How We'll Spend These Three and a Half Hours

*A climb from the easiest terrain to the hardest — with two labs and a demo along the way.*

01

### Greenfield: Spec to Release

35 min

The SDLC pipeline with SDD: architecture, conventions, tests, CI/CD, PR and release readiness.

02

### Lab 1: Greenfield Spec to PR

40 min

Take your Module 04 spec through implementation, tests and CI to a spec-checked pull request.

03

### Brownfield: Archaeology & Impact

30 min

Recover the implicit spec, map dependencies and extension points, size the blast radius.

04

### Lab 2: Brownfield Change

45 min

Analyse an existing app, write a change plan, implement minimally and prove nothing else moved.

05

### App Modernization Patterns

30 min

The modernization spectrum, Strangler Fig, characterization tests, and Copilot's modernization agent.

06

### Migration Demo & Best Practices

30 min

One slice from legacy to modern behind a facade, then the practices that carry across all three terrains.

# Module 05 in the 12-Module Journey

*Module 04 gave you the spec. Now you carry it through the whole lifecycle — on new code, old code and legacy code.*

| FOUNDATION | | | BUILD & VALIDATE | | | EXTEND WITH TOOLS & AGENTS | | | | GOVERN | CAPSTONE |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
| Objectives & GenAI Recap | Copilot Advanced | Prompt & Context Eng. | Spec-Driven Dev (Spec Kit) | **SDD to Full SDLC** | Test Strategy & Validation | MCP & Custom MCP Server | PR Automation & LLM-Judge | No-Code Agents (Rovo) | Code Agents & Simple RAG | Observability, ROI & Gov. | Capstone & Roadmap |

**M04**

The spec → plan → tasks loop is what we now run end to end

**M06**

Spec-to-test traceability becomes a full test strategy

**M07**

MCP connects agents to repos, tickets and build systems

**M08**

PR automation and LLM-judge enforce spec conformance

**Why this matters:** most of your real work is not greenfield. Today's goal is to make AI-assisted change safe in the code you already own.

# Three Terrains, One Habit: The Spec Is the Contract

*What the spec describes — and what can go wrong — changes with the terrain.*

RISK AND EFFORT RISE →

## Greenfield

*Build new*

**SPEC DESCRIBES**

The behaviour you want to create

**MAIN RISK**

Over-building, unclear intent

## Brownfield

*Change existing*

**SPEC DESCRIBES**

What exists today + the delta you want

**MAIN RISK**

Breaking behaviour nobody wrote down

## Modernization

*Replace legacy*

**SPEC DESCRIBES**

Current behaviour + the target to match

**MAIN RISK**

Subtle parity loss, big-bang failure

**SAFETY NET**

Characterization tests + incremental cutover

**Same loop, new inputs:** specify → plan → tasks → implement → verify never changes. What changes is the evidence you gather before the spec and the tests you write before the code.

# Greenfield: The SDD Pipeline from Spec to Release

*Eight stages, left to right. Each produces an artifact and has a human gate; AI accelerates, engineers decide.*




**Key idea:** the same spec drives every stage — the plan, the tests, the PR checklist and the release sign-off all point back to it. That is what “complete SDLC” means in SDD.

# Greenfield Foundations: Architecture and Conventions as Context

*Decide the shape once; let every later prompt inherit it. This is Module 03's LAYER move, applied to a new project.*

**ARCHITECTURE DECISION → plan.md / ADR**

docs/adr/001-layering.md (illustrative)

```
## Decision
Controllers → Services → Repositories.
No controller touches the database.
## Why
Keeps business rules testable without HTTP.
## Consequences
Every feature adds one service + its tests.
```

**CONVENTIONS → .github/copilot-instructions.md**

.github/copilot-instructions.md (illustrative)

```
## Build & test
dotnet build · dotnet test (must pass)
## Code style
Result<T> for expected failures; no exceptions
## Always
Cite the task ID in commit messages
Add a test for every acceptance criterion
```

**Spec says what**

Outcome and acceptance criteria

**ADR says how**

Structure and the reasons for it

**Instructions say always**

Rules every prompt inherits

**Best practice:** write the first ADR and the instruction file before the first feature. Retrofitting conventions after 30 AI-generated files is brownfield work you created yourself.

# Greenfield Quality Gates: Spec → Tests → CI → PR → Release

*Every gate asks the same question in a different place: does this still match the spec?*

**Acceptance tests**
One test per criterion, named after its ID

→

**CI pipeline**
Build, test, lint on every push

→

**PR template**
Links spec, tasks and test evidence

→

**Review**
Reviewer reads spec diff first, then code

→

**Release check**
Every requirement met or consciously deferred

.github/workflows/ci.yml (illustrative)

```yaml
on: [pull_request]
jobs:
  verify:
    steps:
      - run: dotnet build --no-restore
      - run: dotnet test  # fails the PR if red
```

.github/pull_request_template.md (illustrative)

```
- [ ] Spec: specs/001-…/spec.md (changed? Y/N)
- [ ] Tasks done: T001–T008
- [ ] Each AC has a passing test (list IDs)
- [ ] Constitution check: no principle broken
- [ ] AI-generated code reviewed line by line
```

**Best practice:** make the pipeline refuse what the spec forbids. A red CI run on a missing acceptance test teaches more than a style guide.

# Lab 1: Greenfield — Spec to a Checked PR

LAB 1 · 40 min

*Carry your Module 04 feature through implementation, tests, CI and review.*

**Goal** Deliver the Module 04 feature as a pull request where every requirement traces to a task, a test and a passing pipeline.

## STEPS

1. Add the foundations: a one-page ADR and .github/copilot-instructions.md with build/test commands and two rules.
2. Run /speckit.implement phase by phase (fresh chat per phase). Tick a task only when its test passes.
3. Add the CI workflow and the PR template. Push; fix anything the pipeline catches.
4. Open the PR, fill the checklist, and review it yourself against spec.md. Note one gap and fix the spec first.

## FILES YOU CREATE

repo/

```
lab05a/
  docs/adr/001-*.md
  .github/copilot-instructions.md
  .github/workflows/ci.yml
  .github/pull_request_template.md
  specs/001-*/ (from Module 04)
```

## DONE WHEN

The PR shows a green pipeline and a checklist where each acceptance criterion maps to a named, passing test.

# Brownfield Step 1: Repository Archaeology

*Before you can write a change spec, recover what the system already does. Five digs, left to right.*

### 1 Map

Modules, entry points, layers, build and deploy

`architecture notes`

### 2 Trace dependencies

Who calls whom; libraries, services, data stores

`dependency map`

### 3 Recover the as-is spec

Behaviour, rules and contracts as they run today

`as-is spec.md`

### 4 Find extension points

Where the system already expects change

`seams list`

### 5 Note the patterns

Conventions you must preserve

`patterns list`

Ask Copilot to explore — in a subagent or fresh chat — and return a summary with file references. Then verify it by reading the code yourself.

AI summaries of legacy code can sound certain and be wrong. Treat the as-is spec as a hypothesis until a test confirms it.

**Best practice:** keep exploration output small and file-backed (Module 03: SCOPE and COMPRESS). A 1-page as-is spec beats a 40-page chat transcript.

# Brownfield Step 2: Size the Blast Radius

*Change-impact analysis asks: what else could this change touch? Work outward in rings.*




**1 The change**
Files and functions you will edit

**2 Direct dependents**
Callers, subclasses, tests, config that use them

**3 Indirect dependents**
Shared data, events, background jobs, reports

**4 External consumers**
Public APIs, other teams, integrations, contracts

change-impact.md (illustrative)

```
Change: add refund to OrderService
Direct:  OrderController, 14 tests
Indirect: nightly revenue report
External: Billing API v1 (frozen)
Risk: report double-counts refunds
Guard: add report regression test
```

Dependency-analysis tools, IDE “find usages”, and Copilot’s codebase search each find different edges — use more than one, and record what you checked.

**Best practice:** turn each ring into a checklist line with a guard (test, flag or review). Rings 3–4 are where AI-assisted changes usually break things silently.

# Brownfield Step 3: Minimal Change, Maximum Protection

*Prove today's behaviour first, change the least you can, then prove nothing else moved.*

### Pin current behaviour

Write tests that capture what the code does now

### Write a delta spec

Only what changes; reference the as-is spec

### Plan the smallest path

Use existing extension points and patterns

### Implement

Small diffs, one task at a time

### Verify preservation

Old tests still green; new criteria pass

### Preserve patterns

Follow the existing layering and naming — don't introduce a new style in passing

### Keep APIs compatible

Add, don't change; version anything a consumer might depend on

### Stay reversible

Behind a flag or in a small PR you can revert cleanly

**Best practice:** tell Copilot the constraint explicitly: “change only X; do not refactor; keep public signatures”. Left alone, models love to tidy code you did not ask them to touch.

# Lab 2: Brownfield — Add a Feature Safely

LAB 2 · 45 min

*Analyse an existing app, plan the change, implement minimally, prove preservation.*

**Goal** Add one feature to an existing application with a change plan, a delta spec, and evidence that existing behaviour is unchanged.

## STEPS

1. Explore the app (fresh chat or subagent): write a 1-page as-is note — layers, entry points, dependencies, patterns to preserve.
2. Fill change-impact.md using the four rings. Mark every external consumer and the guard you will add for each.
3. Write a short delta spec and plan; pin current behaviour with tests for the code you will touch.
4. Implement with “change only X” instructions. Run all tests, check the diff touches only planned files, open the PR.

## FILES YOU CREATE

```
repo/
lab05b/
  notes/as-is.md
  change-impact.md
  specs/002-delta/spec.md
  tests/ (pinned + new)
```

## DONE WHEN

All pre-existing tests pass, the new criteria pass, and the diff contains no files outside your change plan.

# Modernization: A Spectrum of Ambition

*More change means more value and more risk. Pick the smallest step that solves the real problem.*

AMBITION AND RISK RISE →

## Upgrade

Newer framework or runtime version; fix deprecated APIs

WHEN

**Support ending, security fixes**

RISK Low

## Replatform

Containers or managed cloud services; little code change

WHEN

**Cost, ops, scaling**

RISK Low–Medium

## Re-architect

Split into services or modules; change boundaries and data

WHEN

**Team or scale limits**

RISK High

## Rewrite

New implementation, possibly new language or stack

WHEN

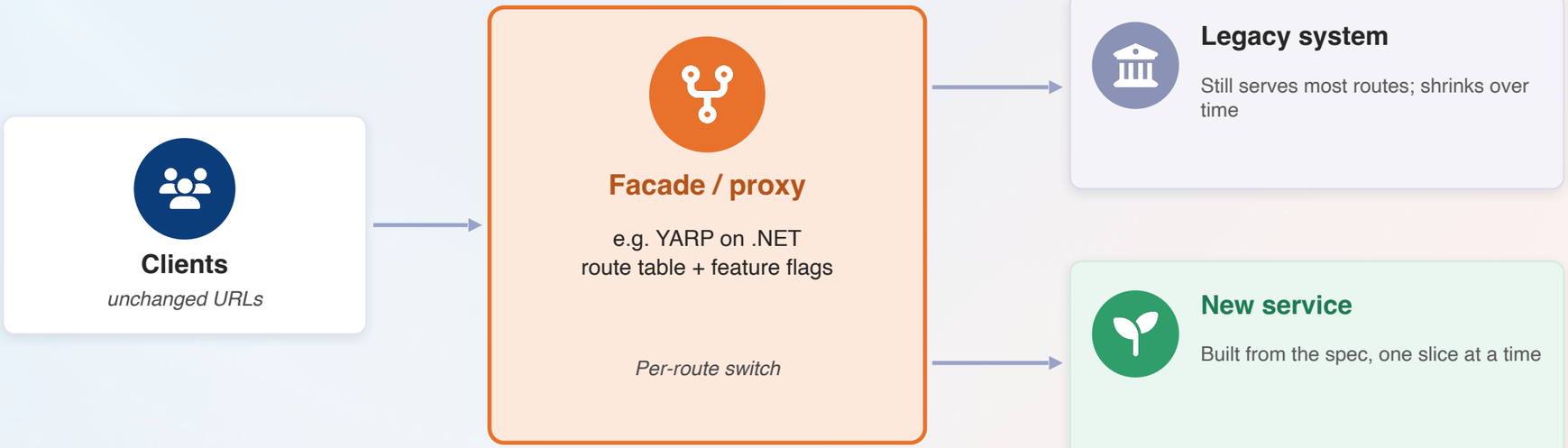
**Last resort**

RISK Highest

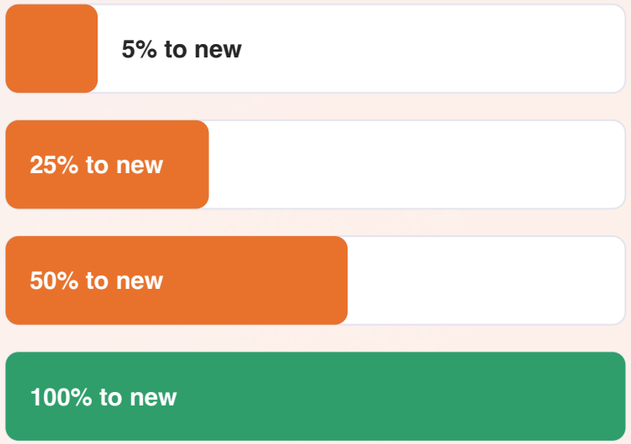
**Reality check:** no general-purpose, GA tool translates one language to another reliably (for example C++ to .NET). AI accelerates each slice; humans own the design and the parity proof.

# The Strangler Fig: Replace One Slice at a Time

*A facade routes each request to legacy or new. Traffic shifts gradually; legacy retires last.*




## EXAMPLE ROLLOUT OF ONE SLICE




*Check errors and latency against baseline at each step; roll back if they drift.*

**Best practice:** retire legacy code only after the new path has run at 100% for at least one release cycle. Until then, the legacy route is your rollback.

# SDD for Modernization: Characterize, Specify, Migrate, Prove

*The legacy system's behaviour becomes the acceptance test for its replacement.*

## THE SDD LOOP FOR ONE SLICE

**Characterize**
Capture legacy outputs as tests

→

**Specify target**
As-is spec + what must differ

→

**Plan slice**
Seam, facade route, data plan

→

**Migrate**
Implement behind a flag

→

**Prove parity**
Same inputs, same outputs

## GITHUB COPILOT MODERNIZATION AGENT — ASSESS → PLAN → EXECUTE → REVIEW

### Assess

Scans code, dependencies and config for outdated frameworks, deprecated APIs and cloud-readiness issues; writes a report to .github/modernize/assessment/

### Plan

Turns the assessment (or your goal, e.g. “upgrade to Spring Boot 3”) into an editable plan.md

### Execute

Applies tasks one by one, validating the build after each change and committing

### Review

You inspect the diff and open a PR — the agent does not replace review

**Best practice:** use the agent’s assessment and plan as inputs to your own spec — edit them. Its output may not always be accurate; keep your characterization tests as the judge.

# Demo: One Slice, Legacy to Modern

*Facilitator-led. Watch the evidence at each step — not just the final working code.*

**1 Pick a slice**

Small, well-bounded route or module with clear inputs and outputs

**2 Characterize**

Run legacy with sample inputs; save outputs as golden tests

**3 Spec + plan**

Spec Kit: as-is behaviour, target stack, facade route

**4 Migrate**

Copilot implements the slice to pass the golden tests

**5 Route + compare**

Flag on: send traffic to new; diff outputs against legacy

## WHERE MIGRATIONS QUIETLY GO WRONG — WATCH FOR THESE

### Memory ownership

Manual allocation and lifetimes have no direct equivalent in managed runtimes — design, don't translate.

### Floating-point parity

Compilers and runtimes can round differently — compare with tolerances you chose on purpose.

### Shared data

Both systems may write the same tables during the transition — decide who owns what.

**Discuss:** which step produced the evidence you would show a sceptical reviewer? (Hint: step 2 and step 5 — the first and last.)

# Best Practices Across the Three Terrains

*Shared habits at the bottom; terrain-specific habits above them.*

## Greenfield

- ADR and instruction file before the first feature
- One test per acceptance criterion
- Pipeline and PR template enforce the spec

## Brownfield

- Recover the as-is spec; treat it as a hypothesis
- Size the blast radius in four rings
- Pin behaviour, change minimally, keep APIs compatible

## Modernization

- Characterization tests before any change
- Strangler Fig: slice, flag, shift, retire last
- Parity proof per slice; avoid big-bang

## ALL TERRAINS

**Spec first, code second** | **Human gate at every stage** | **Small, reversible steps** | **Evidence over assertion** | **Fix the spec, then regenerate**

**Calibrate:** match ceremony to risk. A typo fix needs none of this; a payment-flow change needs all of it.

# What You Can Do Now

*Six outcomes from today — and the habit that ties them together: prove it against the spec.*

- ✓ A greenfield feature is taken from spec to a spec-checked PR through architecture, tests and CI
- ✓ An existing codebase is explored and its implicit spec recovered
- ✓ Change impact is sized in rings, with a guard for each
- ✓ A brownfield change is made minimally, with existing behaviour proven preserved
- ✓ Modernization options are chosen on the spectrum, not defaulted to rewrite
- ✓ A slice is migrated Strangler-Fig style with characterization tests as the judge

**NEXT: MODULE 06 — TEST STRATEGY & VALIDATION · 1.5 HOURS**

Derive a full test strategy from the SDD feature: spec-to-test traceability, mocks and stubs, negative scenarios, and a deliberate defect to prove the tests catch it.

THE CLIMB TODAY

Greenfield → Brownfield → Modernization

# Questions & Discussion

*Module 05 — SDD Across the Full SDLC: Greenfield, Brownfield, Modernization*